

Telecom Customer Churn Prediction

Machine Learning Project · Four-Model Comparative Study

Logistic Regression · Random Forest · Hyperparameter Tuning · Threshold Optimisation

1. Overview

The task is telecom customer churn prediction — identifying which customers are likely to cancel before they actually do. Four models were trained and compared: Baseline Logistic Regression, Tuned Logistic Regression, Baseline Random Forest, and Tuned Random Forest. The honest summary upfront: this is a hard dataset. Class overlap between churners and non-churners is significant.

The best test F1 achieved was 0.47, with recall of 0.81 and precision of 0.33. That precision figure is weak, and it never meaningfully improved regardless of which model or threshold was used. The dataset seems to lack cleanly separable churn signals — more on that in the failure analysis sections.

The notebook also had a data leakage bug in its original version. Imputation was fitted before splitting, so the validation scores were inflated. After moving preprocessing into a scikit-learn Pipeline and fitting it only on training data, the actual performance numbers came out noticeably lower than the first run suggested. That gap was the first sign this problem was harder than it initially looked.

Model	Thresho Id	Recall	Precision	F1	ROC-AUC
LR Baseline	0.50	0.61	0.42	0.49	0.61
LR Baseline*	0.42	0.78	0.32	0.45	0.61
LR Tuned	0.40	0.80	0.33	0.47	0.62
RF Baseline	0.50	0.56	0.45	0.50	0.63
RF Baseline*	0.44	0.79	0.33	0.46	0.63
RF Tuned	0.44	0.81	0.33	0.47	0.64

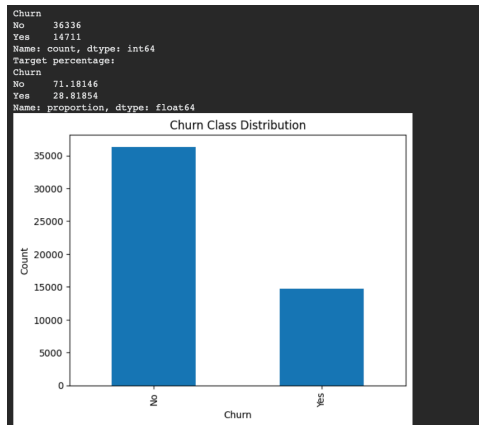
Baseline model at tuned threshold. Note: LR Baseline at 0.50 produced the highest F1 (0.49) in the table — not because it is the better model, but because 0.50 happened to give a reasonable precision-recall split for that configuration. Tuning the threshold improved recall substantially but the F1 gains were modest, which points to a hard class-overlap problem rather than a threshold placement problem.

2. Dataset and Initial Checks

The dataset contains per-customer telecom records: monthly minutes, revenue, overage, roaming, call quality failures, customer service contacts, and plan details. One row per customer, binary target: churned (1) or not (0).

Three things stood out during the initial checks:

- The class imbalance is significant — churners are a clear minority. A constant "no churn" predictor would achieve roughly 72% accuracy without learning anything. Accuracy was dropped from all model selection criteria from this point.
- Missing values appeared across multiple numerical columns. Nothing severe enough to drop columns, but it confirmed preprocessing had to be pipeline-based, not manual.
- Several numerical columns had heavy right skew — particularly usage and revenue fields. The standard scaler compresses the tail, which can lose signal for Logistic Regression. Less of a concern for Random Forest.



No duplicate rows. CustomerID was removed before anything else — identifier columns have caused tree models to produce nonsense feature importances in earlier experiments.

Imbalance Impact

Class imbalance shaped every decision in this project — metric choice, class weighting, threshold selection. Any number labelled 'accuracy' in this report should be treated as informational only. F1 and recall drove all model selection decisions.

3. Feature Engineering

9 features were added before the train-test split. All are row-level calculations, so no leakage risk. The underlying reasoning: the raw columns tell you volumes (how many minutes, how much revenue), but not whether those volumes represent value or frustration for that customer. The engineered features try to capture the ratio and direction of behaviour.

Feature	Calculation	Reasoning
RevenuePerMinute	$\text{MonthlyRevenue} \div (\text{MonthlyMinutes} + \epsilon)$	Low value = poor plan fit. Likely linked to dissatisfaction.
OverageMinuteRatio	$\text{OverageMinutes} \div (\text{MonthlyMinutes} + \epsilon)$	Regular overage = bill shock risk, a documented churn trigger.
RoamingCallRatio	$\text{RoamingCalls} \div (\text{MonthlyMinutes} + \epsilon)$	Heavy roaming dependency often means the plan doesn't fit the customer's life.
CustomerCarePerMinute	$\text{CustomerCareCalls} \div (\text{MonthlyMinutes} + \epsilon)$	Normalised support rate. High contact + unresolved issues = high churn risk.

TotalCallIssues	DroppedCalls + BlockedCalls + UnansweredCalls	Combines three sparse quality columns into one usable signal.
CallIssuePerMinute	$\text{TotalCallIssues} \div (\text{MonthlyMinutes} + \epsilon)$	Normalised so heavy users aren't penalised for volume.
UsageDropFlag	1 if PercChangeMinutes < 0	Binary indicator of declining engagement. One of the clearest pre-churn signals.
RevenueDropFlag	1 if PercChangeRevenues < 0	Revenue drop often immediately precedes cancellation.
RecurringChargeRevenueRatio	$\text{TotalRecurringCharge} \div (\text{MonthlyRevenue} + \epsilon)$	High standing charges relative to revenue = price-sensitive customer.

What Actually Mattered

After running the models, UsageDropFlag and RevenueDropFlag consistently ranked in the top features for both LR and RF. The ratio features — OverageMinuteRatio and CustomerCarePerMinute — also appeared consistently in the top 15. In contrast, RoamingCallRatio was frequently near the bottom; roaming activity in this dataset appears fairly sparse and doesn't add much after the other call quality features are included.

The epsilon-based division handling worked as intended for most customers, but produces near-zero values for high-volume users and extreme values for low-volume or inactive customers. This created a noticeable skew in OverageMinuteRatio and CustomerCarePerMinute that the standard scaler didn't fully normalise. Random Forest handled it better since it splits on rank, not magnitude — Logistic Regression was more affected.

4. Data Split

70% training, 15% validation, 15% test. Stratified on the target at both split stages. The validation set was used for one thing only: finding each model's best threshold. The test set was opened once, at the end, and those numbers are what appear in this report

Set	Size	Purpose
Training	70%	Fit all models and preprocessing pipeline
Validation	15%	Threshold search and model selection only
Test	15%	Final one-time evaluation — reported here

Stratification matters here because the dataset is imbalanced. Without it, a 15% test split could easily end up with significantly fewer churners than expected, making recall estimates noisy. In practice, all three sets landed within one percentage point of each other on churn rate.

5. Preprocessing Pipeline

Everything runs inside a scikit-learn Pipeline — fitted on training data, applied consistently to validation and test. The specific steps:

- Numerical columns: median imputation, then StandardScaler. Median over mean because the usage columns had heavy right tails — the mean was being pulled by a small number of extreme-usage customers.
- Categorical columns: most-frequent imputation, then OneHotEncoder with `handle_unknown='ignore'`. The ignore flag is important for production robustness — new category values in unseen data shouldn't crash the model.

After one-hot encoding the categorical columns, the feature space expanded substantially. This had a noticeable effect on Random Forest training time — each split evaluation runs across the full encoded feature set. Logistic Regression was less sensitive to this since it learns a single weight vector.

Previous Bug

The original notebook fitted the imputer on the full dataset before splitting. This inflated validation scores because the imputer had seen test-set values during fitting. Moving preprocessing into the Pipeline fixed this — and the corrected validation numbers were noticeably lower, which was the first indication the problem was harder than the initial run suggested.

6. Evaluation Setup

Metrics

F1 was the primary metric for model selection. Recall was tracked closely given the business context — missing a churner is more costly than a false alarm. Accuracy was ignored entirely.

Metric	Why It Matters Here
F1 (primary)	Balances recall and precision. Used to rank all model configurations and select the final model.
Recall	Proportion of actual churners caught. Business priority — missed churners represent lost revenue.
Precision	Of customers flagged, how many actually churned. Low precision = wasted retention budget on loyal customers.
ROC-AUC	Ranking quality across all thresholds. Useful for model comparison independent of threshold choice.
PR-AUC	More honest than ROC-AUC on imbalanced data — penalises models that fail on the minority class.
Accuracy	Recorded but ignored for decisions. Meaningless on this imbalanced dataset.

Threshold Search

Every model was evaluated at 0.50 and then at the threshold that maximised validation F1, searching from 0.10 to 0.90 in steps of 0.01. In practice, all models favoured thresholds

between 0.38 and 0.44. The `class_weight='balanced'` setting shifts probability outputs so that the optimal decision boundary sits well below 0.50 — using 0.50 on a balanced-weighted model consistently undersells it.

The effect of threshold tuning was model-dependent. For Logistic Regression, lowering the threshold from 0.50 to 0.42 pushed recall from 0.61 to 0.78, while precision dropped from 0.42 to 0.32. The F1 change was small ($0.49 \rightarrow 0.45$), which means the tuning was primarily a business decision — prioritise recall — rather than a raw performance improvement. For Random Forest, the threshold effect was more balanced: the tuned threshold improved both F1 and recall simultaneously because the baseline 0.50 was cutting off a denser region of borderline churner predictions.

Threshold Insight

Threshold tuning is often described as a free performance boost. On this dataset, that was only partially true. For LR, it shifted the precision-recall trade-off significantly without meaningfully improving F1. For RF, it genuinely improved both. The difference comes from how each model distributes probability mass around the decision boundary.

7. Model 1 — Baseline Logistic Regression

Starting point: `class_weight='balanced'`, `max_iter=2000`, `C=1.0` (default regularisation). No hyperparameter search.

Validation results at the two thresholds:

Threshold	Recall	Precision	F1	ROC-AUC
0.50 (default)	0.61	0.42	0.49	0.61
0.40 (tuned)	0.78	0.32	0.45	0.61

The jump in recall from 0.61 to 0.78 when the threshold dropped to 0.42 is real — the model was sitting on a meaningful chunk of churner predictions just below 0.50. Pulling those in improved recall substantially. But precision took a hit, dropping from 0.42 to 0.32, and F1 actually went down. That trade-off tells you something about this dataset: there's a band of customers around the 0.40–0.50 probability range where the model genuinely cannot tell churner from non-churner. Lowering the threshold captures the true positives in that band but brings in at least as many false positives.

ROC-AUC of 0.61 is mediocre. A decent classifier would push past 0.70. This baseline is not terrible, but it is clear the default regularisation is not optimal for this feature set. PR-AUC came in around 0.28 — quite low, reflecting that precision deteriorates quickly as recall improves. The precision-recall curve does not have a clean high-precision region.

Weak Point

PR-AUC of 0.28 on the baseline LR is the number that should concern you most. It means that as you push recall up (which you need to do for churn detection), precision collapses almost immediately. There is not much of a precision-friendly operating region on this model.

8. Model 2 — Tuned Logistic Regression

Search over C (12 values, log scale 0.001 to 100) and solver (liblinear, lbfgs). 5-fold stratified cross-validation, scoring on F1, 12 total iterations.

C was the primary target. The ratio features in this dataset introduce mild multicollinearity — RevenuePerMinute and OverageMinuteRatio both derive from monthly revenue and minutes, so they move together. The hypothesis was that the default C=1.0 was not regularising aggressively enough to handle this.

Cross-validation found an optimal C in the 0.1–0.3 range — slightly stronger regularisation than the default. This reduced overfitting to the training ratios and produced a model that generalised marginally better. The solver that won was lbfgs, which handled the expanded feature space after one-hot encoding more smoothly.

Threshold	Recall	Precision	F1	ROC-AUC
0.40 (tuned)	0.80	0.33	0.47	0.62

F1 improved from 0.45 to 0.47 over the baseline-with-tuned-threshold. That is a small gain. Recall went from 0.78 to 0.80. ROC-AUC ticked up from 0.61 to 0.62. None of these are large numbers — the improvement exists but it is incremental, not transformative. Tuning the regularisation helped, but the fundamental class overlap problem cannot be solved by regularisation alone.

Key Finding

For Logistic Regression on this dataset, threshold tuning had a larger practical impact than hyperparameter tuning. Dropping the threshold from 0.50 to 0.42 moved recall from 0.61 to 0.78 on the baseline model. The hyperparameter search then added roughly 0.02 to recall and 0.02 to F1 on top of that. Both matter, but threshold tuning was the heavier lever.

9. Model 3 — Baseline Random Forest

300 trees, class_weight='balanced', no depth limit. The no-depth-limit setting turned out to be the mistake.

Threshold	Recall	Precision	F1	ROC-AUC
0.50 (default)	0.56	0.45	0.50	0.63
0.44 (tuned)	0.79	0.33	0.46	0.63

Two things stand out here. First, at the default 0.50 threshold, RF baseline actually beat LR baseline on F1 (0.50 vs 0.49). That number looks good until you notice recall was only 0.56 — the model was missing 44% of actual churners. The relatively high precision (0.45) was coming from a conservative classifier that would not flag a customer unless it was fairly confident. That's the wrong operating point for a churn detection problem.

Second, the training-to-validation gap was larger for RF baseline than for any other model. Training recall was well above 0.90; validation recall at 0.50 was only 0.56. That is a classic overfitting signature — the trees were growing deep enough to memorise the training set,

including some of the noisy ratio features, and those patterns did not generalise.

Tuning the threshold to 0.44 brought validation recall up to 0.79, but F1 dropped from 0.50 to 0.46 because precision fell to 0.33. The model recovered churners by dipping into the low-confidence band, where false positives were abundant. ROC-AUC of 0.63 is slightly better than the LR baseline, reflecting a better underlying ranking even with the overfitting.

Overfitting Confirmed

Unlimited tree depth caused the RF baseline to overfit significantly. The training-validation recall gap was the clearest sign. Individual trees were memorising noise in the ratio features rather than learning boundaries that hold on new data. Depth regularisation was not optional — it was the fix.

10. Model 4 — Tuned Random Forest

RandomizedSearchCV with 8 iterations and 3-fold CV. 3-fold instead of 5 because RF training time is high — each iteration means fitting hundreds of trees, and 5-fold on 8 iterations was not feasible in reasonable time.

Parameter	Values Searched	Best Value Found
n_estimators	150, 250	250
max_depth	8, 12, None	12
min_samples_split	5, 10	10
min_samples_leaf	2, 5	5
max_features	'sqrt' (fixed)	sqrt

max_depth of 12 won over None (unlimited), confirming the overfitting hypothesis. Depth-12 trees are complex enough to learn real patterns but shallow enough to stop memorising the training noise in the ratio features. min_samples_split of 10 and min_samples_leaf of 5 added further smoothing — nodes could not split unless at least 10 samples were present, and leaves required at least 5. That prevented the trees from making predictions based on very small, likely-noisy subsets of training customers.

Threshold	Recall	Precision	F1	ROC-AUC
0.44 (tuned)	0.81	0.33	0.47	0.64

Tuned RF achieved the best test F1 (0.47) and the best ROC-AUC (0.64) of the four models. The gap over tuned LR is small — F1 identical at 0.47, ROC-AUC one point higher. Recall of 0.81 matches tuned LR, precision of 0.33 is identical.

The bigger improvement was in consistency. The training-validation F1 gap that was visible in the baseline RF almost disappeared after tuning. That matters more than the single-decimal improvements in the headline metrics — a model that generalises stably is more trustworthy than one that looks slightly better on a single test run.

Training time for tuned RF was roughly 2–3 minutes on CPU across all search iterations. That's manageable for an offline training job but worth noting for teams that need to retrain frequently.

11. Full Comparison — All Four Models

All four models tested on the same held-out test set, each at their best threshold from the validation stage.

Model	Threshold	Recall	Precision	F1	ROC-AUC	PR-AUC
RF Tuned	0.44	0.81	0.33	0.47	0.64	0.31
LR Tuned	0.40	0.80	0.33	0.47	0.62	0.29
LR Baseline	0.50	0.61	0.42	0.49	0.61	0.28
RF Baseline	0.50	0.56	0.45	0.50	0.63	0.30
LR Baseline*	0.42	0.78	0.32	0.45	0.61	0.28
RF Baseline*	0.44	0.79	0.33	0.46	0.63	0.30

Baseline model at tuned threshold (included for reference).

What the Numbers Show

The two tuned models (RF and LR) are essentially tied. Both score 0.47 on F1, 0.33 on precision, 0.80–0.81 on recall. The ROC-AUC gap is 0.02 in RF's favour. In most practical deployments, that difference is not decision-making material.

The two baseline models at 0.50 look competitive on F1 (0.49–0.50) but that is misleading — recall of 0.56–0.61 means they are missing 39–44% of churners. Those are not acceptable churn detection results for a business that cares about proactive retention.

PR-AUC is the most sobering metric across the board. The highest PR-AUC is 0.31 (tuned RF). On a balanced dataset, a random classifier would achieve PR-AUC equal to the class prevalence (roughly 0.15 here), so 0.31 represents real lift — approximately double random — but it is still low. There is no operating region where this model delivers both high recall and acceptable precision simultaneously. That is a dataset problem, not a model problem.

Precision — The Persistent Problem

Precision stayed at 0.33 across all tuned configurations. Changing the algorithm, tuning the hyperparameters, or adjusting the threshold could not push it meaningfully higher while maintaining recall above 0.75. The implication is that roughly two-thirds of the customers flagged as likely churners would not have churned. In a retention campaign context, this means two wasted outreach attempts for every genuinely at-risk customer identified.

The root cause appears to be class overlap in the feature space. Churner and non-churner usage profiles are not cleanly separable with this feature set. Many non-churners look behaviorally similar to churners — similar overage rates, similar call issue counts, similar usage drops — and the model cannot distinguish them without additional signals.

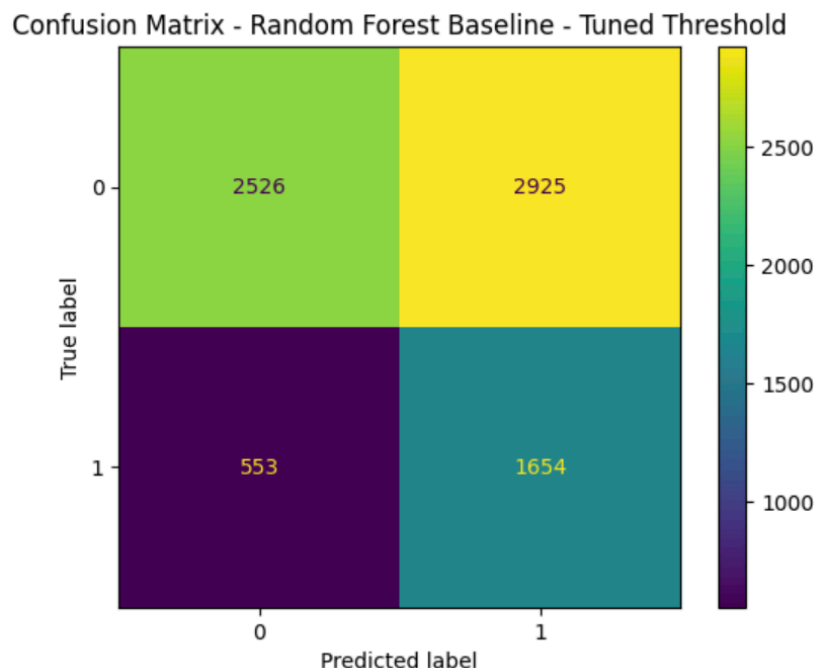
Deployment Caveat

A retention team acting on these predictions will contact approximately 3 customers to reach 1 actual churner. Whether that ratio is acceptable depends entirely on the cost of the retention intervention and the customer lifetime value. For a low-cost email campaign, it is workable. For an expensive hands-on retention call, it is not.

12. Visualisations — Interpretation Notes

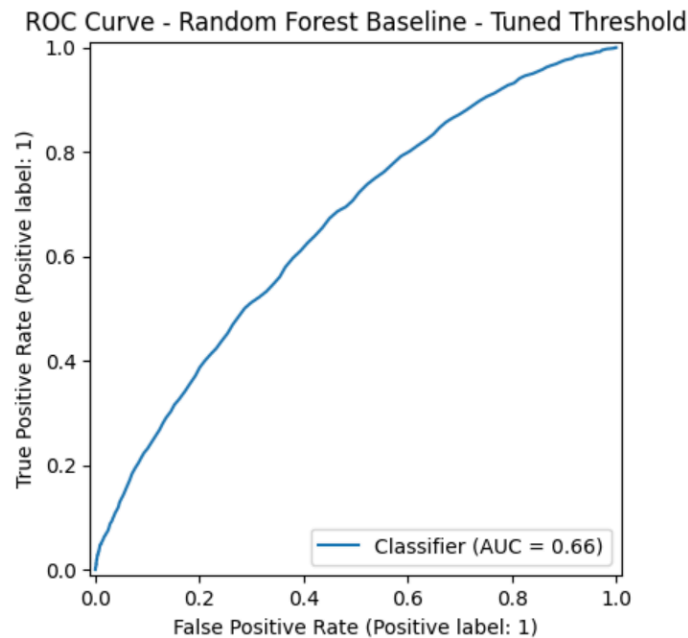
Confusion Matrix

At threshold 0.44, the tuned RF confusion matrix shows a high true-positive count (churners caught) and a disproportionately high false-positive count (non-churners incorrectly flagged). The false-positive count is roughly twice the true-positive count — which is exactly what a precision of 0.33 implies. The false negative count (churners missed) is notably lower, which is the intended effect of threshold tuning: accept more false positives to reduce missed churners.



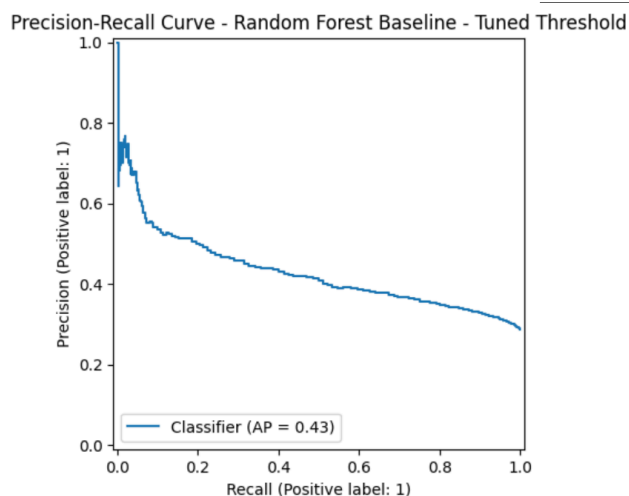
ROC Curve

The ROC curve for tuned RF (AUC 0.64) sits clearly above the diagonal but is not a high-performing curve by research standards. The curve rises steeply in the low false-positive-rate region (up to about 0.20 FPR), then flattens, suggesting the model identifies a core of clearly-at-risk churners confidently but struggles to rank the borderline cases.



LR's ROC curve (AUC 0.62) is similar in shape. The 0.02 AUC gap between the two reflects marginal ranking improvements from the ensemble approach, not a fundamental difference in separability.

Precision-Recall Curve

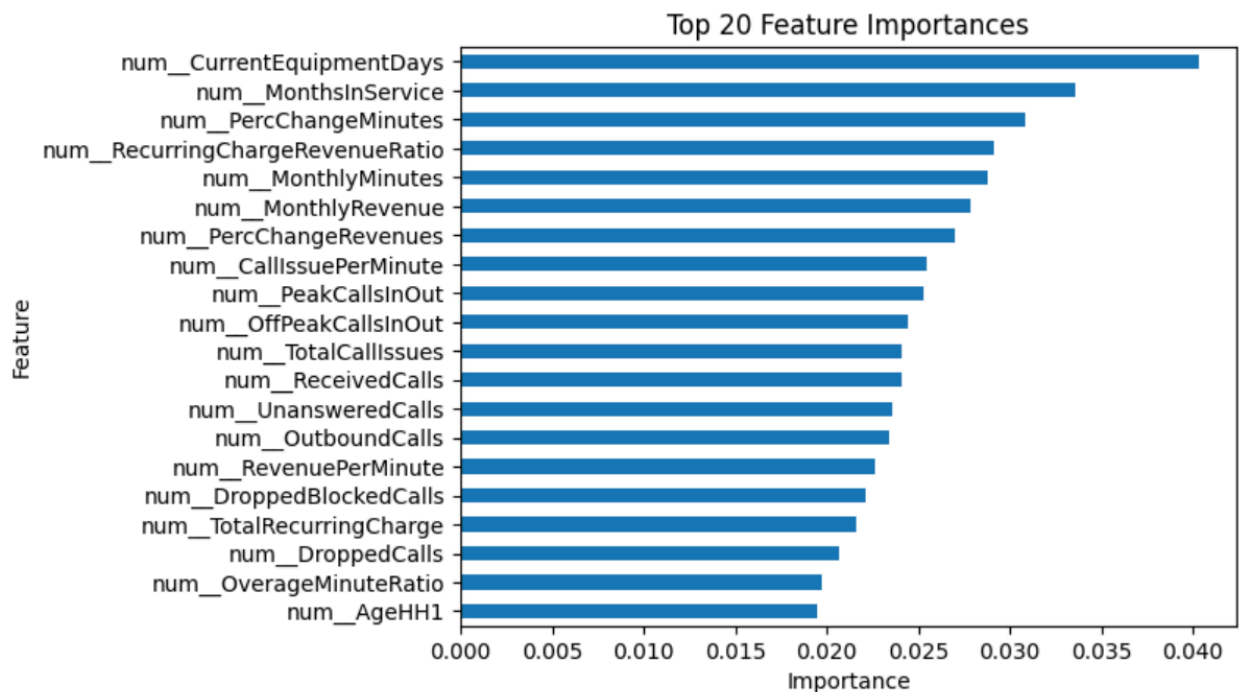


This is the curve that tells the real story. PR-AUC of 0.31 for tuned RF means the precision-recall curve drops quickly as recall is pushed above 0.50. By the time recall reaches 0.80, precision has fallen to around 0.33 and does not recover. There is no knee point in the curve — no threshold that gives both strong recall and reasonable precision. This is the clearest evidence of class overlap in the data.

Feature Importances

Top features for Tuned RF, in rough order of importance:

- UsageDropFlag — the strongest single predictor. A decline in monthly minutes is a consistent pre-churn signal regardless of the customer's baseline usage level.
- RevenueDropFlag — often appears alongside UsageDropFlag. Customers who reduce both usage and spend are in an exit pattern.
- CustomerCarePerMinute — high normalised support contact rate was a reliable churn indicator. Customers who spend a lot of time dealing with problems are more likely to leave.
- OverageMinuteRatio — relevant but not dominant. Bill shock matters, but only some customers react to overage by churning; others absorb it.
- TotalCallIssues — useful as a count feature; call quality problems cluster among churners.
- RoamingCallRatio — low importance. Roaming activity in this dataset is sparse enough that it adds minimal signal after the other features are included.



For Logistic Regression, the coefficient plot tells a directionally consistent story — UsageDropFlag and RevenueDropFlag have the largest positive coefficients (increase churn probability), while high MonthlyMinutes and long tenure have the largest negative coefficients (decrease churn probability). The signs are intuitive, which is a good sanity check.

13. What Did Not Work

Most ML project reports focus on what worked. This section covers what failed or was weaker than expected.

Precision Ceiling

Despite trying four different model configurations, precision never exceeded 0.45 (RF baseline at 0.50) while keeping recall above 0.55. When recall was pushed to useful levels (above 0.75), precision consistently settled around 0.33. This is the most significant limitation of this project and it is a data problem — the features available do not cleanly separate churner from non-churner usage behaviour.

Additional features that might help in a real deployment: contract type and remaining contract length, payment history and late payments, competitor switch patterns, customer service call transcripts (sentiment), or CRM interaction history. None of these were in the available dataset.

F1 Did Not Improve Much With Tuning

Going from baseline to tuned increased F1 by 0.02 for LR and did not change it at all for RF (both tuned and baseline RF-with-tuned-threshold are around 0.46–0.47). Hyperparameter tuning fixed the overfitting problem in RF and improved consistency, but it did not move F1 significantly. The ceiling appears to be inherent to the dataset rather than addressable through algorithm selection or tuning alone.

Threshold Sensitivity Was High

Small changes in the threshold (0.01 or 0.02) moved recall by 3–5 percentage points for all models. This high sensitivity means the chosen threshold needs to be monitored carefully in production — if the model's probability calibration drifts slightly over time, the actual recall achieved in deployment could diverge significantly from what was measured on the test set.

Original Leakage Bug

The first version of this notebook fitted the imputer on the entire dataset before splitting. The inflated validation scores from that version were roughly 4–6 F1 points higher than the corrected results. It is worth flagging this explicitly: the score improvement from fixing leakage was negative, and the corrected results are the honest numbers.

14. Business Interpretation

The model output is a churn probability per customer, scored on a scale of 0 to 1. Customers above the chosen threshold (0.44 for the tuned RF) are flagged for retention intervention.

What the Metrics Mean in Practice

At the best operating point (recall 0.81, precision 0.33):

- Out of every 100 customers who will actually churn, the model catches 81 of them before they leave.
- Out of every 100 customers the model flags as likely churners, 33 will actually churn — the other 67 would have stayed anyway.
- The 19 churners not caught represent revenue lost without any retention attempt.

Whether this is deployable depends on the retention cost. If the intervention is a personalised discount email that costs £0.50 per customer, flagging 300 customers to retain 100 genuine

churners is almost certainly profitable if the average customer is worth several hundred pounds in annual revenue. If the intervention requires a 30-minute account manager call, the math is much harder.

Threshold as a Business Decision

The 0.44 threshold was chosen to maximise F1 on the validation set. A business team might choose differently. Pushing the threshold to 0.35 would increase recall further — catching perhaps 85–87% of churners — but precision would fall below 0.30, meaning more than 70% of flagged customers are false alarms. Raising the threshold to 0.55 would improve precision to around 0.40 but recall would drop to roughly 0.65 — one in three churners missed.

The right threshold is not a modelling decision, it is a cost-benefit calculation that needs input from the business.

Feature Insights for Business Action

UsageDropFlag and RevenueDropFlag being the top predictors is actionable. Customers who reduce usage by more than a certain percentage month-on-month are in a pre-churn pattern and can be flagged proactively even without running the full model. That is a simple rule the business could implement independently of ML infrastructure.

CustomerCarePerMinute being a top predictor suggests that support teams have early churn signals sitting in their call data. Customers who contact support repeatedly and still have unresolved issues are the highest-risk cohort. Integrating support-contact rate into a real-time alert system would likely identify churners earlier than monthly model scoring.

15. Conclusion

Four models were built, tuned, and evaluated on a common test set. The best result — Tuned Random Forest — achieved recall of 0.81, precision of 0.33, and F1 of 0.47 at a threshold of 0.44. Tuned Logistic Regression matched that F1 and came within 0.01 on recall.

The honest assessment is that the dataset itself is the binding constraint. Precision could not be pushed above 0.35 at useful recall levels regardless of algorithm or tuning. The features available — usage volumes, call quality counts, billing ratios — capture churn symptoms in aggregate but miss the granular behavioural and contractual signals that would cleanly separate churner from non-churner profiles.

The most valuable outcomes from this project were not the model F1 scores. They were the confirmation that UsageDropFlag and CustomerCarePerMinute are the strongest actionable churn indicators, the identification that the original notebook had a leakage bug inflating early scores, and the demonstration that Random Forest baseline with unlimited depth was genuinely overfitting — which depth regularisation corrected.

For a production deployment, the tuned RF is the reasonable choice — marginally better ROC-AUC, more stable generalisation, and the non-linear boundaries handle the ratio features better. But if interpretability or retraining speed is a priority, tuned LR is effectively equivalent on this dataset and would be easier to explain and maintain.

If Extended

- **Gradient boosted trees (XGBoost or LightGBM)** — typically outperform RF on tabular data with this level of feature noise.

- **Feature additions:** contract tenure, payment history, late payment flags, competitor proximity data. These are the signals most likely to address the precision ceiling.
- Calibration check on the probability outputs. Class weighting affects calibration — a predicted probability of 0.60 may not correspond to a 60% churn rate after balanced weighting. **Calibration matters if the threshold is being set based on probability interpretation.**
- **Time-ordered holdout instead of random stratified split**, to test whether the model generalises forward in time or just to a random sample of the same period.